

ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

DATA STRUCTURES AND ALGORITHMS

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Data Structures and Algorithms paper 1 · practice a.

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (data structures and algorithms). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept any correct Java solution or method that differs from the model answer. Give marks for correct steps in traces even if a later step is wrong (consistent accuracy). Deduct at most one mark per question for small syntax slips.

QUESTION 1

1.1 B ✓
1.2 C ✓
1.3 A ✓
1.4 B ✓
1.5 B ✓
1.6 B ✓
1.7 B ✓
1.8 B ✓
1.9 C ✓
1.10 D ✓

(10)

[10]

QUESTION 2

2.1 f(n) = O(g(n)) if there are positive constants c and n_0 ✓ such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$ ✓ (2)

2.2 (a) $O(n^2)$ ✓, the inner loop runs n times for each of the n outer passes ✓ (6)
(b) O(log n) ✓, i doubles each time, so the loop runs about $\log_2 n$ times ✓
(c) O(n) ✓, n + 1000 steps and the constant 1000 is ignored ✓

2.3 For $n \ge 1$: $3n^2 + 5n + 2 \le 3n^2 + 5n^2 + 2n^2$ ✓ $= 10n^2$ ✓ (3)
So $c = 10$ and $n_0 = 1$ ✓ (other correct pairs accepted)

2.4 1, log n, n, n log n, n^2, 2^n ✓✓ (one mark if only one pair is out of order) (2)

2.5 n is 4 times bigger, so the time is $4^2 = 16$ times longer ✓: $16 \times 2 = 32$ seconds ✓ (2)

[15]

QUESTION 3

3.1 5 → 3 → 9 ✓✓✓ (8 was removed by removeFirst) (3)

3.2 Model answer: (4)

```
int size() {                                    ✓ heading and return type
    int count = 0;
    Node cur = head;                            ✓ start at head
    while (cur != null) {                       ✓ loop to the end
        count++;
        cur = cur.next;                         ✓ move on and count
    }
    return count;
}
```

3.3 Model answer: (5)

```
void reverse() {
    Node prev = null;                           ✓
    Node cur = head;
    while (cur != null) {                       ✓ visit every node
        Node next = cur.next;                   ✓ remember the rest of the list
        cur.next = prev;                        ✓ turn the link around
        prev = cur;
        cur = next;
    }
    head = prev;                                ✓ new head is the old last node
}
```

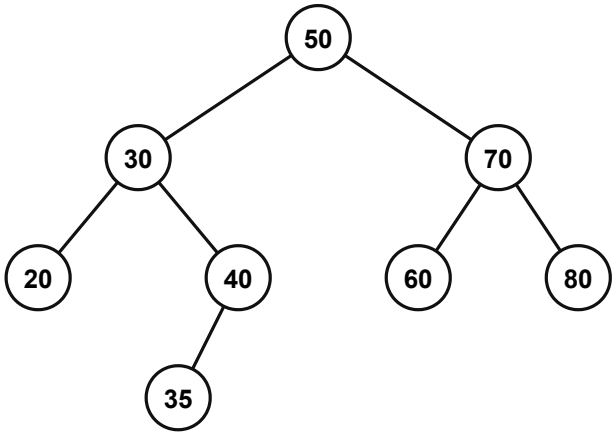
3.4 6: [6] 2: [6, 2] 3: [6, 2, 3] ✓ (4)
+: [6, 5] ✓ *: [30] ✓ 4: [30, 4] −: [26]
Result 26 ✓

3.5 Positions 0 to 4: F, B, C, D, E ✓✓✓ (A was dequeued and F wrapped around to position 0) (4)
front = 1 ✓

[20]

QUESTION 4

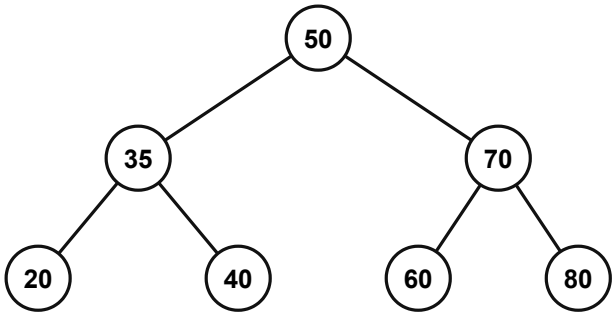
4.1



Correct root and level 2 ✓; 20, 40, 60, 80 correct ✓✓; 35 is the left child of 40 ✓ (4)

4.2 Preorder: 50 30 20 40 35 70 60 80 ✓✓ (6)
Inorder: 20 30 35 40 50 60 70 80 ✓✓
Postorder: 20 35 40 30 60 80 70 50 ✓✓

4.3



35 replaces 30 ✓; 20 and 40 are the children of 35 ✓; 40 no longer has a left child ✓ (3)

4.4 3 (50 → 30 → 40 → 35) ✓ (1)

4.5 A self-balancing binary search tree in which the heights of the two subtrees of every node differ by at most 1 ✓. This keeps the height O(log n), so search, insert and delete stay O(log n) ✓ (2)

4.6 Model answer: (4)

```
int countLeaves(TreeNode t) {
    if (t == null) return 0;                                  ✓ empty tree
    if (t.left == null && t.right == null) return 1;      ✓ a leaf
    return countLeaves(t.left) + countLeaves(t.right);     ✓✓ both subtrees added
}
```

[20]

QUESTION 5

5.1 Pass 1: [3, 7, 9, 1, 5] ✓ (4)
Pass 2: [3, 7, 9, 1, 5] ✓
Pass 3: [1, 3, 7, 9, 5] ✓
Pass 4: [1, 3, 5, 7, 9] ✓

5.2 [5, 1, 11, 15, 61, 37, 59, 26] ✓✓✓ (4)
The pivot 15 is at index 3 ✓

5.3 Merge sort copies the halves into temporary arrays to merge them, so it needs O(n) extra space ✓; quick sort partitions by swapping inside the same array, needing only the recursion stack ✓ (2)

5.4 0: 28 1: 22 2: 15 3: 10 4: 31 5: 4 6: 17 (5)
10, 22 and 28 placed directly ✓; 31 → 4 ✓; 4 → 5 ✓; 15 → 2 ✓; 17 probes 3, 4, 5 and goes to 6 ✓

5.5 7 / 7 = 1 (the table is full) ✓ (3)
Primary clustering: keys that collide fill neighbouring slots, forming long runs ✓, so later keys that hash anywhere into the run need many probes ✓

5.6 Any ONE: separate chaining, each slot holds a linked list of all keys that hash there ✓✓; quadratic probing, probe h, h+1², h+2², …; double hashing, a second hash function gives the step size (2)

[20]

QUESTION 6

6.1 A, B, C, D, E, F ✓✓✓ (3)

6.2 A, B, D, E, F, C ✓✓✓ (3)

6.3 An adjacency list ✓: it stores only the edges that exist, O(V + E) space, while a matrix always uses O(V²) space ✓ (2)

6.4 P, Q, R, S, T (or P, R, Q, S, T) ✓✓ (2)

6.5 AC 1 ✓, BC 2, DE 2 ✓, EF 3 ✓, AB 4 rejected (it would make the cycle A–B–C) ✓, BD 5 Total weight = 1 + 2 + 2 + 3 + 5 = 13 ✓ (5)

[15]

TOTAL: 100

END OF MARKING GUIDELINE